

1. What is Express.js

- **Definition:** Minimal and flexible Node.js web application framework.
- **Purpose:** Build web servers, REST APIs, middleware support, route handling.

Basic Example:

```
const express = require('express');
const app = express();
const PORT = 3000;

app.use(express.json());

app.get('/', (req, res) => {
  res.send('Hello World from Express.js!');
});

app.listen(PORT, () => {
  console.log(`Server running at http://localhost:${PORT}`);
});
```

2. MVC (Model-View-Controller)

- **Definition:** Architectural design pattern to organize code into three main components.

Components:

1. **Model**
2. Handles data and business logic.
3. Manages database operations or in-memory data.
4. **View**
5. Handles user interface.
6. Displays data to users using templates (EJS, Pug) or frontend frameworks (React, Angular).
7. **Controller**
8. Handles user requests.
9. Interacts with Model to fetch/update data.
10. Passes data to View for rendering.

Analogy to Remember MVC:

- **Restaurant Analogy:**
- **Model** = Kitchen (prepares the food/data)
- **View** = Dining Table (presents food/data to customers/users)
- **Controller** = Waiter (takes orders, talks to kitchen, serves to table)
- This analogy helps remember how requests flow and responsibilities of each component.

MVC Example with Express.js:

```
// model/userModel.js
const users = [];
module.exports = users;

// controller/userController.js
const users = require('../model/userModel');
exports.getUsers = (req, res) => { res.json(users); };
exports.addUser = (req, res) => { users.push(req.body);
res.status(201).json(req.body); };

// routes/userRoutes.js
const express = require('express');
const router = express.Router();
const userController = require('../controller/userController');
router.get('/', userController.getUsers);
router.post('/', userController.addUser);
module.exports = router;

// app.js
const express = require('express');
const app = express();
const userRoutes = require('./routes/userRoutes');
app.use(express.json());
app.use('/users', userRoutes);
app.listen(3000, () => console.log("Server running on http://localhost:3000"));
```

3. Express.js Core Features

1. **Fast & Minimal Framework**
2. **Routing** (GET, POST, PUT, DELETE)
3. **Middleware Support**
4. **Template Engine Support** (EJS, Pug, Handlebars)
5. **HTTP Utilities** (`req`, `res` helpers)
6. **Static File Serving**

- 7. REST API Support
- 8. Scalability & Flexibility
- 9. Third-party Middleware Ecosystem (morgan, cors, helmet)
- 10. Error Handling

Example Middleware:

```
app.use((err, req, res, next) => {  
  console.error(err.stack);  
  res.status(500).send('Something broke!');  
});
```

4. HTTP Methods in Express.js Routing

Method	Purpose	Example
GET	Fetch data	/users
POST	Create new data	/users
PUT	Update entire resource	/users/:id
PATCH	Update part of resource	/users/:id
DELETE	Remove resource	/users/:id
ALL	Matches all HTTP methods	/secret
USE	Middleware / route grouping	/api

Example GET/POST:

```
app.get('/users', (req, res) => res.send('Get all users'));  
app.post('/users', (req, res) => res.send('Create new user'));
```

5. Sending Data in Requests: Body, Query, Params

1. JSON Body (req.body)

- Sent in **POST/PUT/PATCH** request body.
- Requires `express.json()` middleware.

```
app.post('/users', (req, res) => {
  console.log(req.body); // { name: 'Alisha', age: 22 }
  res.send(`User ${req.body.name} added`);
});
```

2. Query Parameters (req.query)

- Sent in URL after `?` for GET requests.

```
app.get('/search', (req, res) => {
  console.log(req.query); // { city: 'Bangalore' }
  res.send(`Search results for ${req.query.city}`);
});
// URL: /search?city=Bangalore
```

3. Route Parameters (req.params)

- Dynamic parts of URL, defined with `:`.

```
app.get('/users/:id', (req, res) => {
  console.log(req.params); // { id: '101' }
  res.send(`Get user with ID ${req.params.id}`);
});
```

Type	Location	Access	Use Case
Body	Request body	<code>req.body</code>	Create/update data
Query	URL after ?	<code>req.query</code>	Filtering/searching
Params	URL path	<code>req.params</code>	Identify resource

6. Why POST Responses Don't Show in Browser

- **Browsers send GET requests by default** when entering URL.
- POST requires **body data**, which browser address bar cannot send.
- **Solutions:**
 - HTML form submission with `method='POST'` and `action='/users'`
 - JavaScript `fetch()` / AJAX
 - Use tools like **Postman/Thunder Client**

Example Fetch POST in Browser:

```
fetch('http://localhost:3000/users', {  
  method: 'POST',  
  headers: { 'Content-Type': 'application/json' },  
  body: JSON.stringify({ name: 'Alisha', age: 22 })  
})  
.then(res => res.json())  
.then(data => console.log(data));
```

Summary: - Express.js = Node.js framework for web servers & APIs. - MVC = Structure for Model, View, Controller, easy to remember via Restaurant analogy. - Core features include routing, middleware, REST APIs, template engines. - HTTP methods: GET, POST, PUT, PATCH, DELETE, ALL. - Data sent via `req.body`, `req.query`, `req.params`. - POST requests require body; browsers can't send POST directly via URL.

End of Express.js & MVC Guide